UNIVERSIDAD COMPLUTENSE DE MADRID

FACULTAD DE ESTUDIOS ESTADÍSTICOS

Minería de datos y Modelización Predictiva



Machine Learning – Clasificación Binaria

Juan Peñas Utrilla

Máster Big Data, Data Science & Inteligencia Artificial

Curso académico 2025-26

Índice

1. Introducción 3
2. Análisis exploratorio del dataset 3
3. Matriz de crrelaciones 9
4. Análisis de componentes principales PCA 10
5. Reanálisis con 2 componentes principales 12
6. Determinación del número de grupos 14
7. Calcular las predicciones y los intervalos de confianza 18
8. Comparación de las predicciones de los modelos 20

1. Introducción

El presente trabajo aborda uno de los desafíos más críticos y representativos en el sector de las telecomunicaciones: la predicción del abandono o fuga de clientes (*Customer Churn*). Utilizando un conjunto de datos que detalla el comportamiento de uso, la información geográfica y las características de los planes contratados por los usuarios, el objetivo principal es desarrollar un modelo predictivo capaz de identificar anticipadamente qué clientes tienen una alta probabilidad de dar de baja su servicio. Resolver este problema de clasificación binaria no solo representa un interesante reto analítico, sino que aporta un valor de negocio incalculable, permitiendo a las compañías diseñar campañas de retención proactivas y optimizar sus recursos.

Para lograr este objetivo, se ha implementado un flujo de trabajo de *Machine Learning* riguroso, robusto y orientado a la puesta en producción. La metodología abarca desde un preprocesamiento exhaustivo —que incluye la transformación de variables categóricas, el escalado de datos numéricos y la selección de las características más relevantes para mitigar el ruido temporal— hasta el modelado predictivo avanzado. En concreto, el núcleo de la experimentación se centra en el diseño y ajuste de Redes Neuronales Artificiales (Perceptrón Multicapa). A través de técnicas de validación cruzada y búsqueda exhaustiva de hiperparámetros (*GridSearchCV*), se busca encontrar el equilibrio perfecto entre la complejidad matemática del modelo y su capacidad de generalización, garantizando métricas sólidas sin caer en el sobreajuste.

2. Análisis y depuración de los datos

Se comienza importando el dataset, como los nombres de las columnas son arbitrarias siendo desde V1 hasta V20 y la columna objetivo Y, se cambian los nombres para visualizar mejor las variables:

```
df = pd.read_csv('BBDD_ML_TAREA.csv')
df.head()
rename_dict = {"V1": "state", "V2": "account_length","V3": "area_code",
    "V4": "phone_number","V5": "international_plan", "V6": "voice_mail_plan",
    "V7": "number_vmail_messages", "V8": "total_day_minutes","V9": "total_day_calls",
    "V10": "total_day_charge", "V11": "total_evening_minutes", "V12": "total_evening_calls",
    "V13": "total_evening_charge","V14": "total_night_minutes", "V15": "total_night_calls",
    "V16": "total_night_charge", "V17": "total_international_minutes", "V18":
"total_international_calls",
    "V19": "total_international_charge", "V20": "customer_service_calls", "Y": "y"}
df = df.rename(columns=rename_dict)
```

	state	account_length	area_code	phone_number	international_plan	voice_mail_plan	number_vmail_messages	total_day_minutes	total_day_calls	total_day_charge
0	43	121	415.0	1118	0	0	0	86.1	100	14.64
1	7	170	510.0	3326	0	0	0	184.1	106	31.30
2	31	96	510.0	2146	0	0	0	150.0	122	25.50
3	17	90	415.0	387	0	0	0	193.7	83	32.93
4	7	61	415.0	313	0	0	0	140.6	89	23.90

5 rows × 21 columns

Fig. 1 Vista general del dataset (VER SI ES MEJOR PONER UNA TABLA O ALGO)

Para obtener una visión general de los datos se ejecutan las siguientes líneas de código:

```
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9200 entries, 0 to 9199
Data columns (total 21 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   state                           9200 non-null   int64
 1   account_length                  9200 non-null   int64
 2   area_code                       9125 non-null   float64
 3   phone_number                    9200 non-null   int64
 4   international_plan              9200 non-null   int64
 5   voice_mail_plan                 9200 non-null   int64
 6   number_vmail_messages           9200 non-null   int64
 7   total_day_minutes               9200 non-null   float64
 8   total_day_calls                 9200 non-null   int64
 9   total_day_charge                9174 non-null   float64
 10  total_evening_minutes           9200 non-null   float64
 11  total_evening_calls             9200 non-null   int64
 12  total_evening_charge            9200 non-null   float64
 13  total_night_minutes             9128 non-null   float64
 14  total_night_calls               9140 non-null   float64
 15  total_night_charge              9200 non-null   float64
 16  total_international_minutes     9200 non-null   float64
 17  total_international_calls       9200 non-null   int64
 18  total_international_charge      9156 non-null   float64
 19  customer_service_calls          9200 non-null   int64
 20  y                               9200 non-null   int64
dtypes: float64(10), int64(11)
memory usage: 1.5 MB
```

	state	account_length	area_code	phone_number	international_plan	voice_mail_plan	number_vmail_messages	total_day_minutes	total_day_calls	total_
count	9200.000000	9200.000000	9125.000000	9200.000000	9200.000000	9200.000000	9200.000000	9200.000000	9200.000000	
mean	26.232391	102.230326	437.141370	2512.690978	0.175326	0.217826	6.517826	191.584424	100.207826	
std	14.697631	40.065555	42.320277	1438.955632	0.380266	0.412791	12.872573	61.523538	20.247057	
min	0.000000	1.000000	408.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	14.000000	75.000000	415.000000	1253.000000	0.000000	0.000000	0.000000	148.100000	87.000000	
50%	26.000000	102.000000	415.000000	2536.500000	0.000000	0.000000	0.000000	189.300000	100.000000	
75%	39.000000	128.000000	510.000000	3751.250000	0.000000	0.000000	0.000000	236.700000	114.000000	
max	50.000000	243.000000	510.000000	4999.000000	1.000000	1.000000	52.000000	351.500000	160.000000	

8 rows × 21 columns

Fig. 2 Información general del dataset

En la figura 2 se aprecia que todas las variables presentan valores numéricos, algunas variables toman valores discretos (int64) y otros continuos (float64). Además se ve que hay como mínimo una variable ('area_code')(Poner área code como otra categoría ya que puede ser que no se tenga esa información debido a vergüenza a responder) que presenta valores perdidos. A continuación se analizan los valores perdidos presentes en el dataset para ver si se pueden imputar o hay que eliminar registros o incluso variables.

```
info_nulos = pd.DataFrame({ 'Total Nulos': df.isna().sum(),
                            'Porcentaje': df.isnull().mean()*100})
print(info_nulos[info_nulos['Total Nulos'] > 0].sort_values(by='Porcentaje', ascending=False))
```

```
                           Total Nulos  Porcentaje
area_code                           75    0.815217
total_night_minutes                 72    0.782609
total_night_calls                   60    0.652174
total_international_charge          44    0.478261
total_day_charge                    26    0.282609
```

Fig. 3 Resumen de valores perdidos

Se observa en la figura 3 que hay 7 variables con valores perdidos, el porcentaje de valores perdidos respecto del número total de registros es muy pequeño, inferior siempre al 1%. Se va a analizar si son valores perdidos de manera aleatoria o tienen alguna correlación.

```
plt.figure(figsize=(10,3))
sns.heatmap(correlacion_nulos, annot=True, cmap='coolwarm', vmin=-1, vmax=1, fmt=".2f")
```

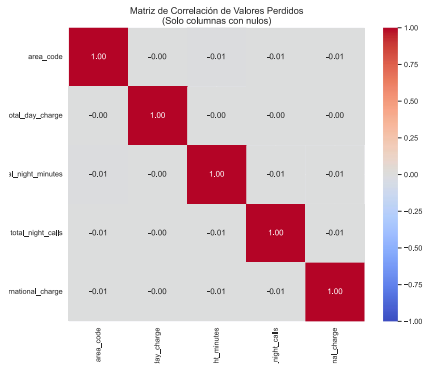


Fig. 4 Heatmap de valores perdidos

Como los valores perdidos no siguen ninguna relación se procede a imputarlos usando un KNNImputer, de esta manera se imputan las variables en base a los vecinos más próximos de cada registros con perdidos, en este caso se seleccionan los 5 vecinos más cercanos.

```
from sklearn.impute import KNNImputer
imputer = KNNImputer(n_neighbors=5)
X = df.drop(columns=['y'])
X_new = (imputer.fit_transform(X))
X_new = pd.DataFrame(X_new, columns=X.columns, index=df.index)
df_imputed = pd.concat([X_new, df['y']], axis=1)
```

Tras usar el código ya no quedan valores perdidos por lo que se puede comenzar a realizar el análisis exploratorio de los datos. Primero se va a analizar la distribución de la variable objetivo:

```
df_imputed['y'].value_counts()
```

```
y
0    4600
1    4600
Name: count, dtype: int64
```

Fig. 4 Heatmap de valores perdidos

En la figura 5 se observa que el problema de clasificación no es desbalanceado pudiendo afrontar el problema de forma normal. Según la descripción del enunciado y la información de la figura 2 se tienen 3 tipos de variables además de la target:

```
cat_cols = ['area_code', 'international_plan', 'voice_mail_plan']
num_discretas_cols = ['customer_service_calls',
total_international_calls','number_vmail_messages']
target = 'y'
num_cols = [col for col in df_imputed.columns if col not in cat_cols + num_discretas_cols +
['y']]
col_sin_target = [col for col in df_imputed.columns if col != 'y']
```

PONER AQUÍ 4 COUNTPLOTS DEL ANEXO y explicar un poco las variables

En la figura se aprecia que la variable phone_number no discrimina la variable objetivo, además la variable es única por usuario, es decir, hace como de identificador por lo que se decide eliminarla del dataset para que no entre en el modelado.

```
df_imputed = df_imputed.drop(columns = ['phone_number'])
```

Una vez se tienen una idea sobre los datos se analizan las crrelaciones que hay entre las variables para mirar si hay variables con una correlación muy elevada y así evitar la multicolinealidad.

```
correlacion = df_imputed.corr()
plt.figure(figsize=(20,10))
sns.heatmap(correlacion, annot = True)
plt.savefig('figuras/heatmap_correlacion_lineal.svg')
plt.show()
```

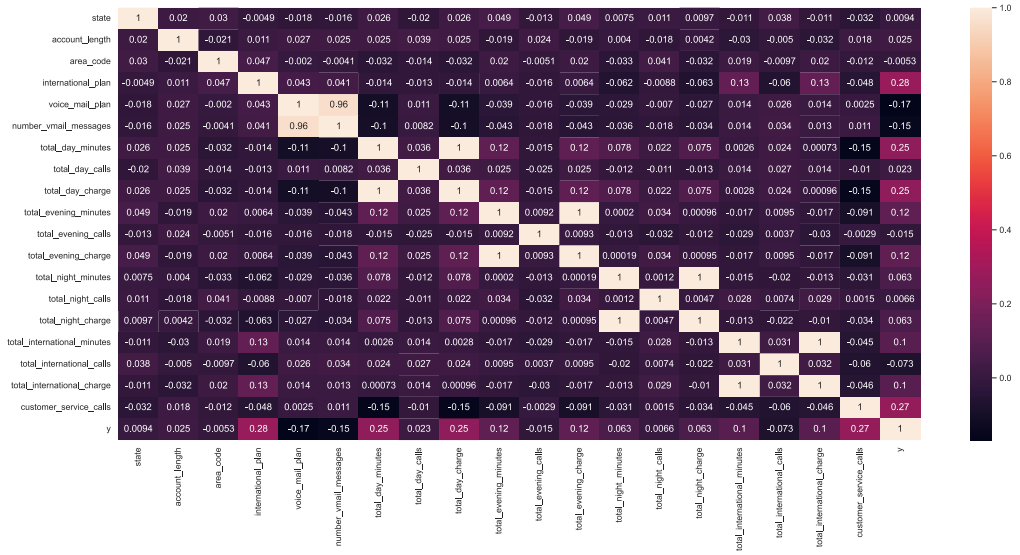


Fig. 3 Estadísticas descriptivas de las variables del dataset

En la figura se observa como hay variables con una correlación del 100%, esto quiere decir que esas variables aportan la misma información y se puede eliminar una de los pares.

```
import numpy as np
num_cols = [c for c in df_imputed.columns if c not in cat_cols ]
# Solo numéricas
corr_matrix = df_imputed[num_cols].corr().abs()

# Triángulo superior (para evitar duplicados)
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
plt.figure(figsize=(20,10))
sns.heatmap(upper, annot = True)
plt.show()
# Variables a eliminar
threshold = 0.9
to_drop = [col for col in upper.columns if any(upper[col] > threshold)]

# Nuevo dataframe
df_reduced = df_imputed.drop(columns=to_drop)

print("Variables eliminadas:", to_drop)
```

Se han eliminado (variables) y ahora se tienen 15 variables el siguiente paso es comenzar por realizar un proceso de RFE para después entrenar una red neuronal.

Se representan diferentes gráficos para entender mejor los datos, para las variables categóricas se representan boxplots y para las numéricas diagramas de dispersión indicando en la leyenda la variable ‘species’:

```
fig, axes = plt.subplots(len(num_vars), 2, figsize=(14, 3.5*len(num_vars)), sharey='row')
for i, var in enumerate(num_vars):
    # Columna 0: island
    sns.boxplot(data=penguins, x="island", y=var, hue="species", palette=palette, ax=axes[i,
0])
    sns.swarmplot(data=penguins, x="island", y=var, hue="species", palette=palette,
                  dodge=True, alpha=0.6, size=3, ax=axes[i, 0])
    axes[i, 0].set_title(f"{var} — island")
    axes[i, 0].legend_.remove()
```

```
    # Columna 1: sex
    sns.boxplot(data=penguins, x="sex", y=var, hue="species", palette=palette, ax=axes[i, 1])
    sns.swarmplot(data=penguins, x="sex", y=var, hue="species", palette=palette,
                  dodge=True, alpha=0.6, size=3, ax=axes[i, 1])
    axes[i, 1].set_title(f"{var} — sex")
    axes[i, 1].legend_.remove()
# Leyenda común
handles, labels = axes[0, 0].get_legend_handles_labels()
fig.legend(handles, labels, title="species", loc="upper right")
plt.tight_layout(rect=[0, 0, 0.92, 1])
plt.savefig("categorical_pairplot_4x2.svg")
plt.show()
```

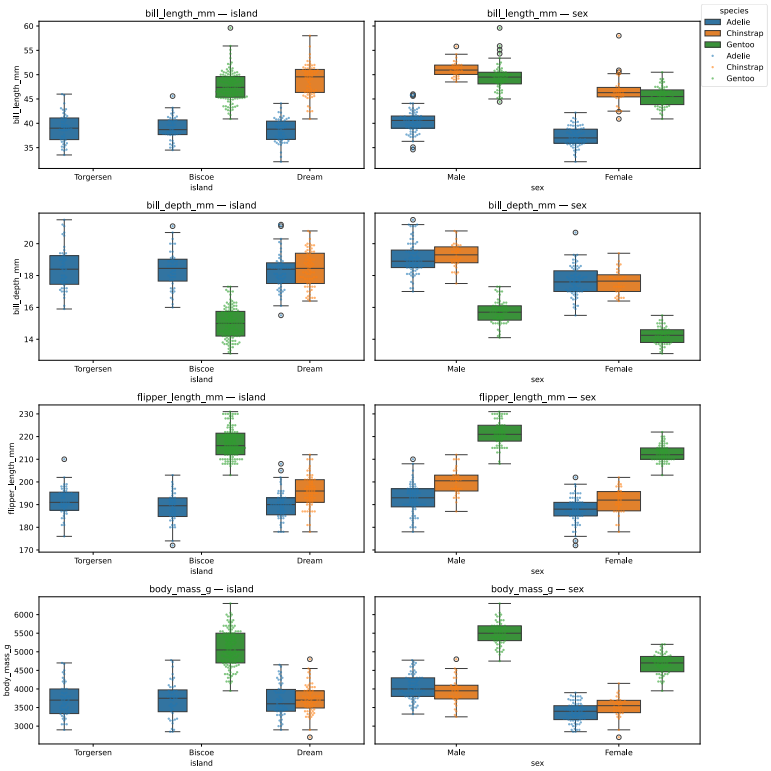


Fig. 3 Boxplots de las variables categóricas en función de las numércias

También se realizan gráficos de dispersión para las continuas:

```
fig_1 = sns.pairplot(data=penguins, hue="species")
fig_1.savefig("pairplot_penguins.svg", format="svg")
```

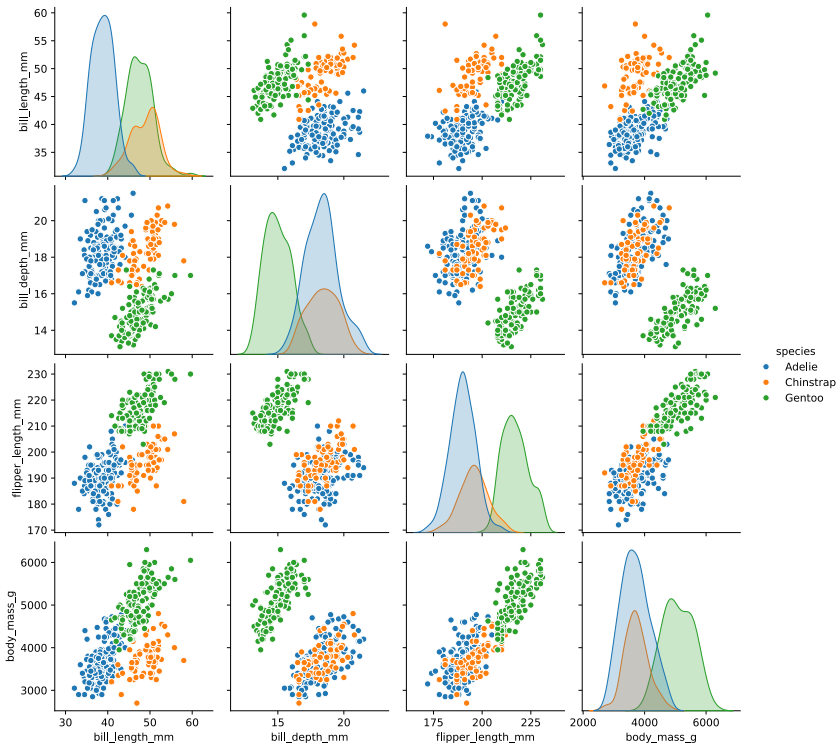


Fig. 4 Boxplots de las variables categóricas en función de las numércias

Adicionalmente, se ha realizado un pairgrid:

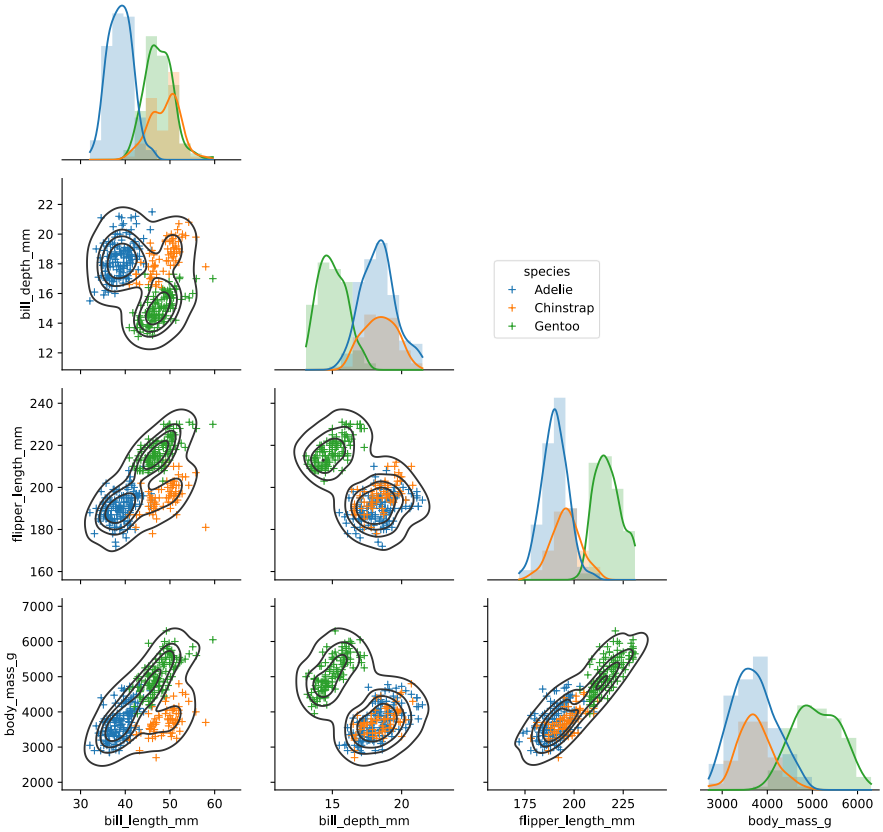


Fig. 5 Pairgrid variables numércias

En las Figuras 3, 4 y 5 se presentan los análisis descriptivos de las variables del conjunto de datos, considerando como variable objetivo la especie (*species*). En primer lugar, se observa que en la isla Torgersen únicamente aparecen individuos de la especie Adelie, lo que indica una distribución geográfica no uniforme de las especies en el dataset.

En cuanto a las diferencias por sexo, en promedio, los machos presentan valores superiores a las hembras en las variables morfológicas analizadas, independientemente de la especie considerada.

El análisis de los gráficos de dispersión y del *PairGrid* muestra que la especie Gentoo presenta un subgrupo claramente diferenciado, con una separación visual relativamente nítida respecto a las otras especies en varias combinaciones de variables. Por el contrario, las especies Adelie y Chinstrap muestran un mayor solapamiento en el espacio de características, lo que dificulta su separación visual y anticipa una clasificación más compleja entre ambos grupos.

Finalmente, en la Figura 5 se observa que en los gráficos de dispersión que incluyen bill_length en el eje horizontal, las tres especies tienden a agruparse en regiones diferenciadas, aunque persiste la presencia de individuos con características similares entre especies, lo que indica que ninguna variable aislada permite una separación perfecta de las clases.

3. Matriz de crrelaciones

Primero se va a calcular la matriz de correlaciones entre las variables numéricas y se va a representar la matriz:

```
# Calcula la matriz de correlaci ´on entre las variables del DataFrame notas

correlacions = penguins[["bill_length_mm", "bill_depth_mm",

    "flipper_length_mm","body_mass_g"]].corr()

# Crea una nueva figura de dimensiones 10 x8 para el gr´a fico .

plt.figure(figsize =(10 , 8) )

sns.heatmap(correlacions, cmap="vlag", vmin=-1, vmax=1,

             center=0, annot=True, fmt=".2f", linewidths=0.5)

plt.savefig("corr_matrix.svg", format="svg")

plt.show()
```

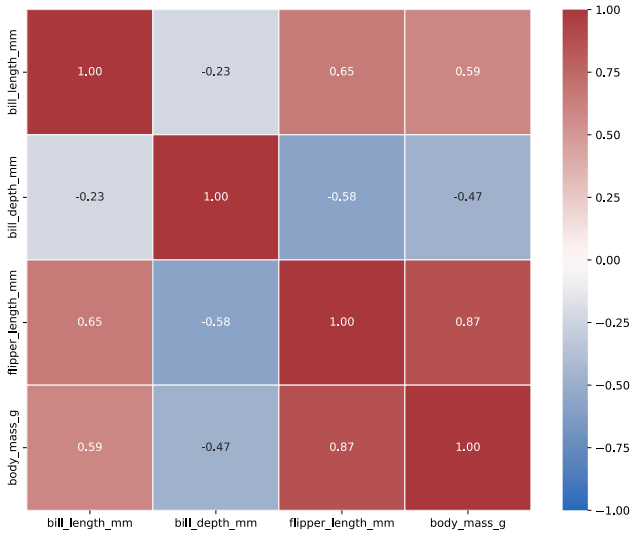


Fig. 6 Matriz de correlación de las variables numéricas

En la Figura 6 se muestran las correlaciones entre las variables numéricas del conjunto de datos. Se observa una correlación positiva elevada entre la longitud de las aletas (flipper_length_mm) y la masa corporal (body_mass_g), lo que indica que los pingüinos de mayor tamaño tienden a presentar una mayor masa. Asimismo, la longitud del pico (bill_length_mm) muestra una correlación positiva moderada tanto con la longitud de las aletas como con la masa corporal.

Por el contrario, la profundidad del pico (bill_depth_mm) presenta correlaciones negativas con la longitud de las aletas (−0.58) y con la masa corporal (−0.47), lo que sugiere que, a medida que aumentan estas variables asociadas al tamaño corporal, la profundidad del pico tiende a disminuir.

4. Análisis de componentes principales PCA

Para realizar el análisis es necesario estandarizar las variables numéricas, ya que estas son las únicas que entran en el análisis.

```
num_vars = ["bill_length_mm", "bill_depth_mm",
    "flipper_length_mm","body_mass_g"]
penguins_std = StandardScaler().fit_transform(penguins[num_vars])
```

Una vez estandarizados se puede entrenar el algoritmo, estudiar los autovalores y las gráficas.

```
pca = PCA(n_components=4)
fit = pca.fit(penguins_std)
display(fit)
# Obtener los autovalores asociados a cada componente principal.
autovalores = fit.explained_variance_
```

```
# Obtener la varianza explicada por cada componente principal como un porcentaje de
la varianza total.

var_explicada = fit.explained_variance_ratio_*100

# Calcular la varianza explicada acumulada a medida que se agregan cada componente
principal.

var_acumulada = np.cumsum(var_explicada)

# Crear un DataFrame de pandas con los datos anteriores y establecer índice.

data = {'Autovalores': autovalores, 'Variabilidad Explicada': var_explicada,
'Variabilidad Acumulada': var_acumulada}

tabla = pd.DataFrame(data, index=['Componente {}'.format(i) for i in range(1,
fit.n_components_+1)])

# Imprimir la tabla

display(tabla.style.format("{:.2f}"))

plot_varianza_explicada(var_explicada, fit.n_components_)
```

	Autovalores	Variabilidad Explicada	Variabilidad Acumulada
Componente 1	2.75	68.63	68.63
Componente 2	0.78	19.45	88.09
Componente 3	0.37	9.22	97.30
Componente 4	0.11	2.70	100.00

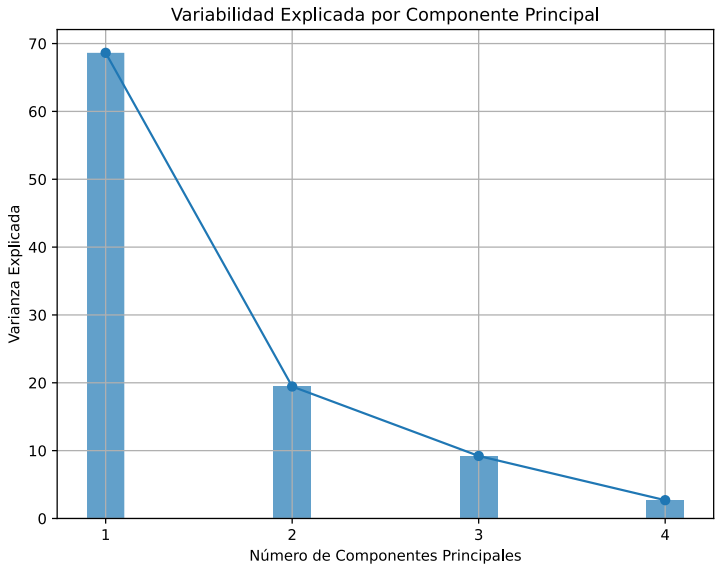


Fig. 7 Autovalores obtenidos y variabilidad por componente principal

El análisis de la varianza explicada muestra un claro punto de inflexión tras la segunda componente principal. La primera componente explica por sí sola el 68.6 % de la variabilidad, mientras que la inclusión de la segunda eleva la varianza explicada acumulada hasta el 88.1 %. A partir de este punto, la contribución de las componentes restantes es residual, lo que justifica la reducción dimensional a dos componentes como un compromiso adecuado entre conservación

de información y simplicidad del modelo.

5. Reanálisis con 2 componentes principales

Se va a proceder a analizar como quedan los datos de pingüinos al aplicar las dos componentes principales con las que nos hemos quedado.

Primero se va a graficar los autovectores en el nuevo espacio 2D de componentes principales.

```
plot_corr_circle(correlaciones_notas_con_cp)
```

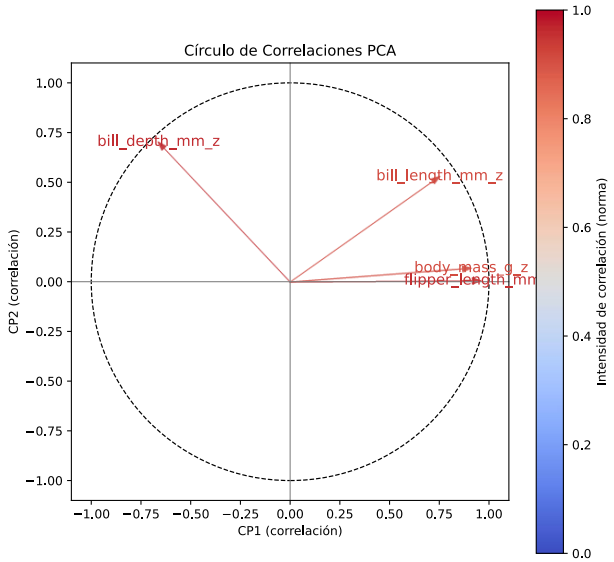


Fig. 8 Autovectores con las características físicas de los pinguinos

En la Figura 8 se muestra el círculo de correlaciones correspondiente al análisis de componentes principales, donde se representan las proyecciones de las variables físicas sobre el plano definido por las dos primeras componentes. Se observa que la componente principal 1 está fuertemente asociada con la masa corporal y la longitud de las aletas, además de la longitud del pico, todas ellas con contribuciones positivas y elevadas. Esto indica que dicha componente puede interpretarse como un eje relacionado con el tamaño corporal global del pingüino.

Por su parte, la componente principal 2 presenta una estructura menos dominante y está principalmente influenciada por la profundidad del pico (*bill_depth_mm*), que aparece en dirección opuesta a las variables asociadas al tamaño. Este comportamiento sugiere que la segunda componente recoge variaciones morfológicas más específicas del pico, diferenciando individuos con mayor profundidad de pico frente a aquellos caracterizados por mayores dimensiones corporales generales.

Ahora se representan todos los registros sobre el subespacio de las PCA:

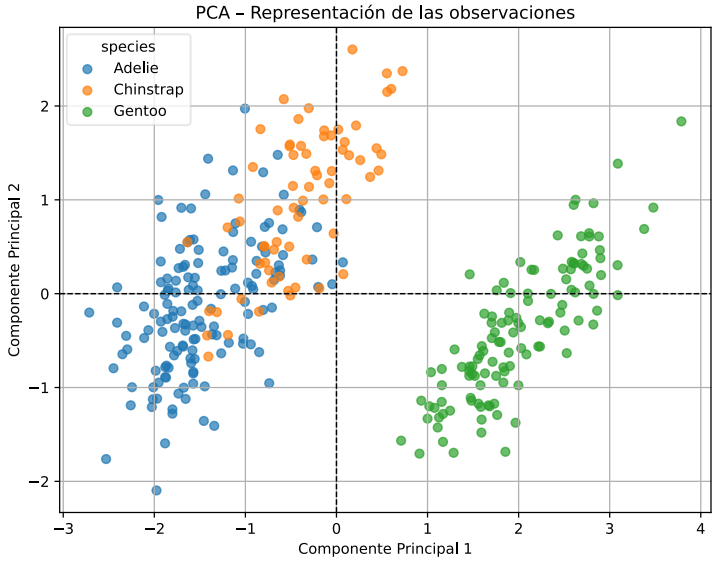


Fig. 9 Todos los registros sobre el subespacio de las PCA

En la Figura 9 se representa la distribución de los individuos en el plano definido por las dos primeras componentes principales. Se observa una clara separación de la especie Gentoo respecto a las otras dos especies a lo largo de la primera componente principal. Los individuos Gentoo presentan valores elevados en la componente 1, mientras que Adelie y Chinstrap se sitúan mayoritariamente en valores negativos, lo que refleja las diferencias en tamaño corporal capturadas por esta componente.

Las especies Adelie y Chinstrap muestran una mayor proximidad en el espacio PCA, con cierto solapamiento entre sus individuos. No obstante, la segunda componente principal introduce una diferenciación adicional entre ambas: los individuos Chinstrap tienden a presentar valores positivos en esta componente, mientras que los Adelie se concentran alrededor de valores cercanos a cero o ligeramente negativos. A pesar de esta tendencia, la separación entre estas dos especies no es completa, lo que indica la presencia de características morfológicas similares en algunos individuos.

A la hora de construir un índice que valore de forma conjunta las características de los pingüinos, se tiene que hacer uso del siguiente código:

```
loadings = pd.Series(pca.components_[0], index=num_cols).sort_values(key=np.abs,
ascending=False)
print(loadings)
```

```
flipper_length_mm    0.576825
body_mass_g          0.549675
bill_length_mm       0.453753
bill_depth_mm       -0.399047
dtype: float64
```

Fig. 10 Pesos de las variables físicas estandarizadas a la PC1

Reescribiendo en una expresión los pesos de la figura 10 quedaría de la siguiente manera:

$$\text{PC1} = 0.58\, z_{\text{flipper_length_mm}} + 0.55\, z_{\text{body_mass_g}} + 0.45\, z_{\text{bill_length_mm}} - 0.40\, z_{\text{bill_depth_mm}}$$

Donde z_i indica las variables estandarizadas. Se observa que la componente principal se puede expresar como una combinación lineal de las variables estandarizada, con mayores contribuciones positivas de la longitud de las aletas, la masa corporal y la longitud del pico, mientras que la profundidad del pico presenta una contribución negativa. En conjunto, esta componente puede interpretarse como un índice global asociado al tamaño corporal del pingüino, ya que agrupa las variables relacionadas con mayores dimensiones físicas.

Para saber el índice de cada especie se puede utilizar el siguiente código:

```
indice_por_especie = scores.groupby("species")["PC1"].mean()
print(indice_por_especie.round(2))
```

```
species
Adelie      -1.46
Chinstrap   -0.39
Gentoo       2.01
Name: PC1, dtype: float64
```

Fig. 11 Índices de cada especie de la PC1

En la Figura 11 se muestran los índices globales obtenidos para cada especie, calculados como la media de los valores de la primera componente principal de todos los individuos pertenecientes a cada grupo. Este valor medio representa la posición característica de cada especie sobre el eje definido por la componente principal 1. Tal y como se ha descrito previamente, la especie Gentoo presenta un índice positivo elevado (aproximadamente 2.01), claramente diferenciado del resto. Por su parte, las especies Adelie y Chinstrap toman valores negativos, aunque Adelie se sitúa más desplazada hacia valores bajos (−1.46) que Chinstrap (−0.39), lo que refleja diferencias relativas en sus características morfológicas dentro del espacio PCA.

6. Determinación del número de grupos: Agrupamiento jerárquico

Para determinar el numero de grupos que se pueden formar en el dataset de penguins se va a obtener un clustermap, reorganizando las filas y columnas según a agrupación jerárquica, de esta manera se puede estimar o comenzar a intuir el posible número de grupos. Antes de comenzar con este análisis es fundamental que todas las variables cuantitativas tengan la misma escala, ya que de otra manera aquellas con un orden de magnitud mayor dominan en las medidas de similitud entre individuos, para ello se utiliza el siguiente código:

```
num_vars = ["bill_length_mm", "bill_depth_mm",
    "flipper_length_mm","body_mass_g"]
penguins_std = StandardScaler().fit_transform(penguins[num_vars])
```

Después se obtiene el clustermap:

```
palette = {"Adelie":"tab:blue","Chinstrap":"tab:orange","Gentoo":"tab:green"}
row_colors = penguins.loc[penguins_std.index,"species"].map(palette)
g = sns.clustermap(penguins_std, cmap="coolwarm", row_colors=row_colors)
```

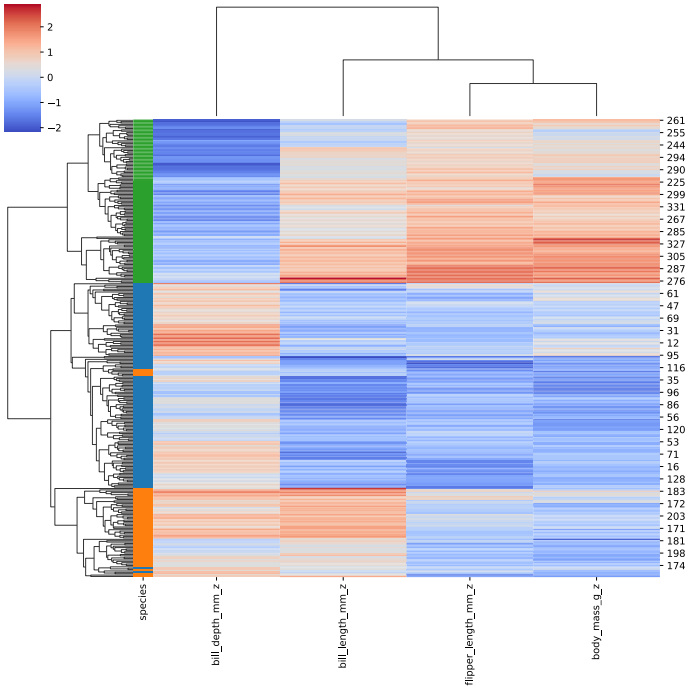


Fig. 12 Clustermap con la especie de cada pingüino

En la figura 12 se observa el clustermap de los pingüinos, en el lado izquierdo aparecen las especies de cada pingüino, se aprecia que se han reordenado los registros, apareciendo bien separados los de la especie Gentoo (color verde) y mezclados las otras dos especies Adelie (azul) y Chinstrap (naranja).

Ahora se calcula la matriz de distancias, para ello se utilizan los datos estandarizados y se emplea la métrica euclídea, se muestra un pequeño ejemplo con 5 pingüinos:

```
# Calculate the pairwise Euclidean distances
distance_matrix = distance.cdist(penguins_std, penguins_std, 'euclidean')
# The distance_matrix is a 2D array containing the Euclidean distances
# between all pairs of observations.
print("Distance Matrix:")
distance_small = distance_matrix[:5, :5]
#Index are added to the distance matrix
distance_small = pd.DataFrame(distance_small, index=penguins_std.index[:5],
columns=penguins_std.index[:5])
distance_small_rounded = distance_small.round(2)
print(distance_small_rounded)
```

```
Distance Matrix:
      0     1     2     3     4
0  0.00  0.76  1.25  1.08  1.17
1  0.76  0.00  1.00  1.28  1.66
2  1.25  1.00  0.00  0.98  1.47
3  1.08  1.28  0.98  0.00  0.88
4  1.17  1.66  1.47  0.88  0.00
```

Fig. 13 Matriz de distancias para los 5 primeros registros

Ahora que se tiene una visión general del proceso, se va a graficar la matriz de distancias de todo el dataset en un mapa de calor:

```
df_distance = pd.DataFrame(distance_matrix, index = penguins_std.index, columns =
penguins_std.index)
sns.heatmap(df_distance, annot=False, cmap="YlGnBu", fmt=".1f")
```

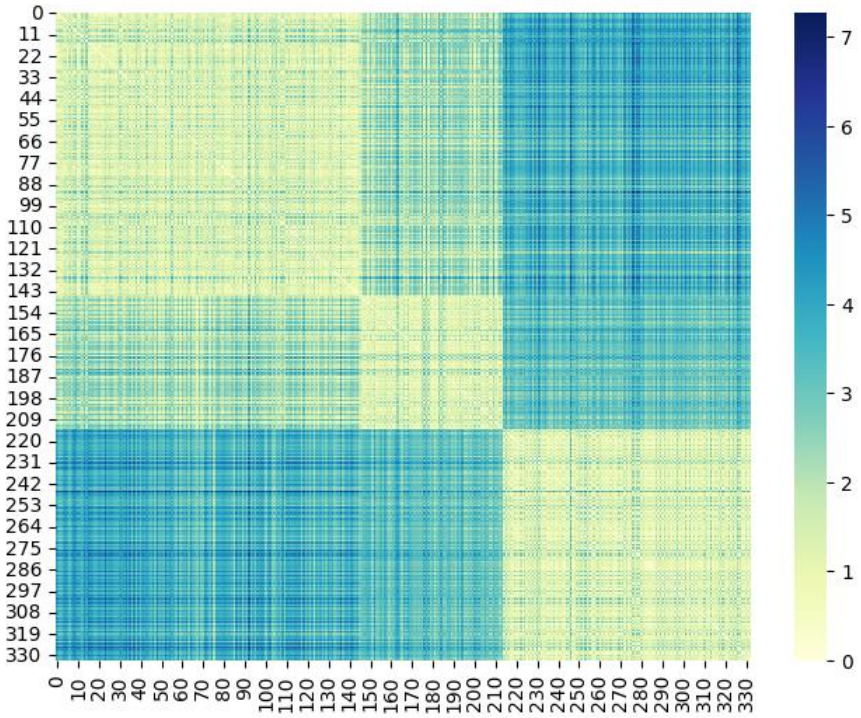


Fig. 14 Matriz de distancias en heatmap

La Figura 14 revela un patrón distintivo caracterizado por la formación de dos grupos claramente diferenciados. Los cuadrados blancos representan individuos con una alta similitud (distancias próximas a 0). Por otro lado, en el cuadrante superior izquierdo se identifican dos subgrupos cuya diferenciación es más sutil debido a su mayor grado de semejanza.

También se puede obtener un heatmap reordenando los registros en función de su similitud se pueden utilizar diferentes métodos, en este trabajo se han empleado el 'average' y el 'ward':

```
# Perform hierarchical clustering to get the linkage matrix
linkage = sns.clustermap(df_distance, cmap="YlGnBu", fmt=".1f", annot=False,
method='average').dendrogram_row.linkage
# Reorder the data based on the hierarchical clustering
order = pd.DataFrame(linkage, columns=['cluster_1', 'cluster_2', 'distance',
'new_count']).index
reordered_data = penguins_std.reindex(index=order, columns=order)
# Optionally, you can add color bar
sns.heatmap(reordered_data, cmap="YlGnBu", fmt=".1f", cbar=False)
plt.show()
```

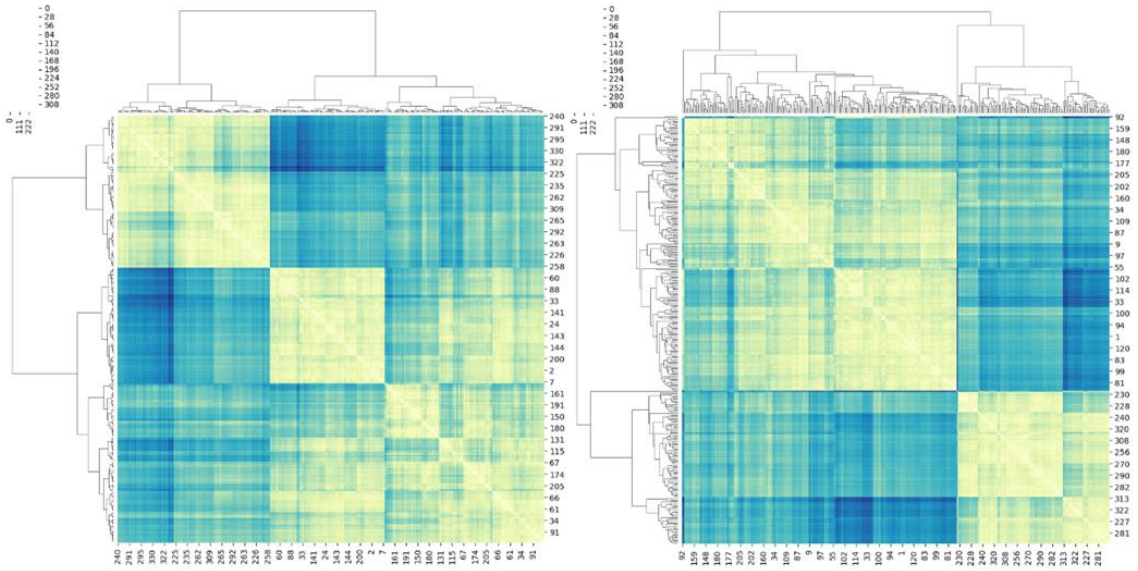


Fig.16 Heatmap con los datos reordenados a la izquierda average a la derehca ward

En la figuras 16 se confirma las observaciones previas al mostrar tres grupos definidos, advirtiendo que dos de ellos muestran una alta similitud.

Se llevará a cabo la construcción de un dendrograma con el objetivo de determinar el número óptimo de clústeres y validar la estructura jerárquica presente en los datos:

```
# Calculate the linkage matrix            method = single
linkage_matrix = sch.linkage(df_distance, method='ward')
# Create the dendrogram
dendrogram = sch.dendrogram(linkage_matrix, labels=penguins_std.index,
leaf_font_size=9, leaf_rotation=90)
plt.axhline(y=150, color='r', linestyle='--')   # corte para 3 clusters
# Display the dendrogram
plt.show()
```

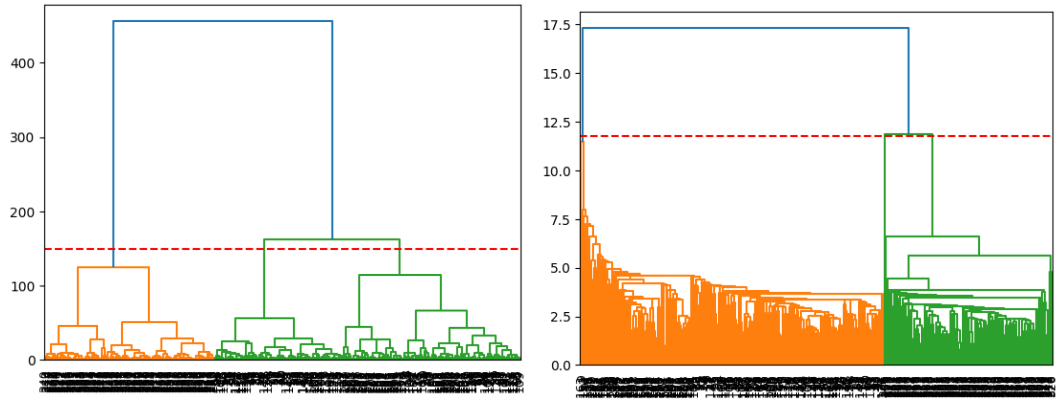


Fig.16 Dendogramas a la izquierda ward a la derecha single

Se generaron dendrogramas empleando los métodos de Single Linkage y Ward, coincidiendo ambos en una estructura de entre 2 y 3 clústeres. Se optó por la solución de 3 grupos para preservar una mayor riqueza informativa. Cabe destacar que, aunque en el método Single la estabilidad para 3 grupos es limitada, sugiriendo que una partición de 2 o incluso 4 grupos sería más robusta, se mantuvo el criterio de 3 para asegurar la consistencia entre ambos análisis.

Con los 3 grupos seleccionados se asigna a cada individuo un grupo en base a sus características.

```
num_clusters = 3
cluster_assignments = sch.fcluster(linkage_matrix, num_clusters,
criterion='maxclust')
penguins_std['Cluster4'] = cluster_assignments
```

Y ahora se representa en el espacio de las PCA los grupos hechos con el análisis de agrupamiento jerárquico:

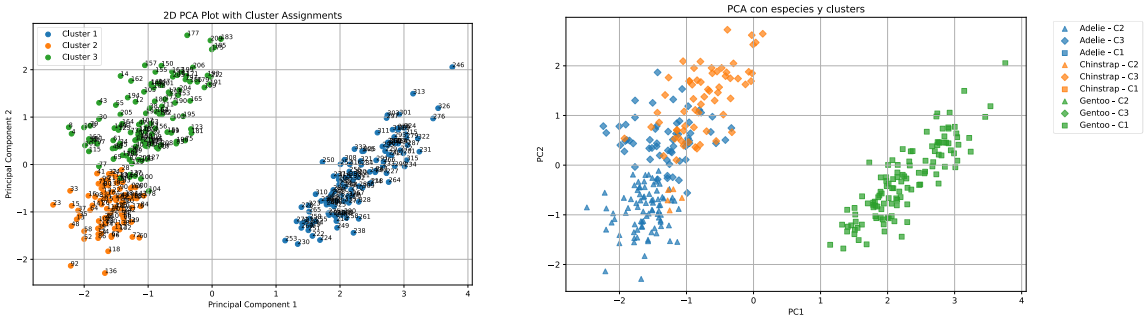


Fig.17 Agrupamientos de los individuos en el espacio de PCA

El algoritmo de agrupamiento jerárquico identificó con éxito una estructura de tres clústeres, aunque con distintos niveles de precisión. Se observa en la figura 17 que el algoritmo fue capaz de aislar perfectamente a la especie Gentoo, asignándole un grupo exclusivo y bien diferenciado del resto (Clúster 1), lo que indica una evidente diferencia entre esta especie y las demás.

Por el contrario, la separación entre las especies Adelie y Chinstrap resultó más compleja para el método jerárquico. Si bien el algoritmo intentó dividir a estos individuos en dos grupos distintos (Clústeres 2 y 3), la visualización revela que existe una frontera difusa entre ellos. Esto sugiere que las distancias fenotípicas entre Adelie y Chinstrap son muy cortas, provocando que el algoritmo tenga dificultades para establecer un corte limpio y resulte en una mezcla de individuos en los grupos asignados.

7. Agrupamiento K-Means

Tras el análisis previo, se procede a aplicar el algoritmo de K-Means. Para la ejecución del modelo, se ha definido inicialmente un valor de k=3, fundamentado en la estructura de tres grupos identificada mediante el agrupamiento jerárquico. Para ello se utiliza el siguiente código:

```
# Set the number of clusters (k=3)
k = 3
# Initialize the KMeans model
kmeans = KMeans(n_clusters=k, random_state=0)
# Fit the KMeans model to your standardized data
kmeans.fit(penguins_std)
# Get the cluster labels for your data
kmeans_cluster_labels = kmeans.labels_
print(kmeans_cluster_labels)
```

```
[2 2 2 2 0 2 0 2 0 0 2 2 0 2 0 2 2 2 2 2 2 2 2 2 2 2 2 2 2 0 0 2 2 0 2 0
 2 0 2 0 2 2 0 2 0 2 0 2 0 2 0 2 0 2 2 2 0 2 0 2 0 2 0 2 0 2 0 2 2 2 2 0
 2 0 2 2 2 0 2 2 2 2 2 2 0 2 0 2 0 2 0 2 0 2 0 2 0 2 2 2 0 2 0 2 0 2 0 0 0 2
 0 2 2 2 2 2 2 0 2 0 2 0 2 0 2 0 2 0 2 0 2 0 2 0 2 2 2 2 2 2 2 0 2 2 2 2 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 0 2 0 0 0 0 0 0 0 0 0 0 0 2 0 2 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]
```

Fig.18 Cluster de cada pingüino según el algoritmo k-means

Tras la ejecución del algoritmo, se obtiene la asignación de clústeres para cada individuo de la muestra. Con el fin de validar visualmente la coherencia de esta agrupación, se proyectan nuevamente las observaciones sobre el espacio bidimensional de las componentes principales, codificando cada punto con un color distintivo según el grupo asignado por el modelo K-means.

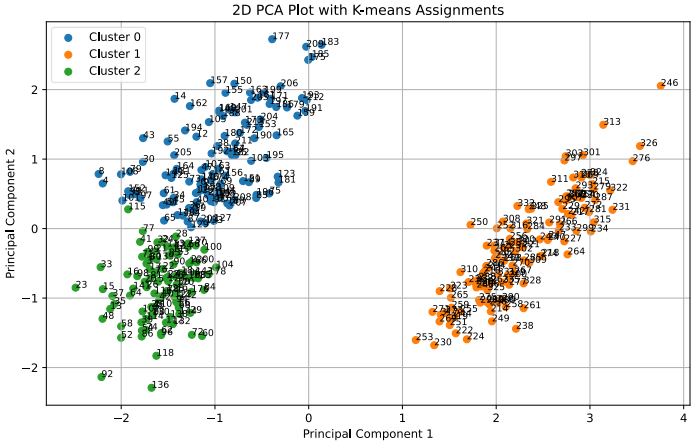


Fig.19 Agrupaciones de los pingüinos según k-means en el espacio de las PCA

Con el fin de validar estadísticamente la idoneidad del número de clústeres seleccionados, se ha procedido a calcular la suma de los cuadrados entre conglomerados (WCSS) para diferentes valores de k. Este análisis, conocido como el método del codo, permite evaluar la disminución de la varianza interna a medida que aumenta la complejidad del modelo, ayudando a identificar el punto óptimo de partición.

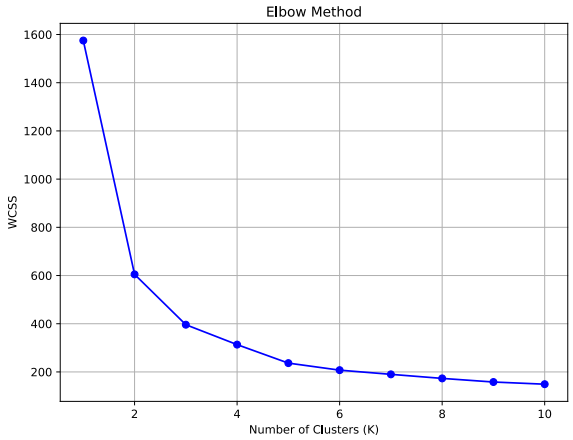


Fig.20 Evaluación de WCSS según el número de grupos k

A partir de la inspección de la curva del método del codo (Figura 20), se determina que la partición óptima corresponde a k=3. Se observa un punto de inflexión en este valor, a partir del cual el descenso de la suma de cuadrados intragrupo (WCSS) disminuye notablemente, lo que indica que añadir clústeres adicionales no aporta una mejora significativa en la cohesión de los grupos.

8. Validación del agrupamiento

Como complemento al método del codo, se emplea la puntuación de silueta (Silhouette Score) para validar la calidad del agrupamiento. Esta métrica evalúa la cohesión de los puntos dentro de su propio clúster en comparación con su separación respecto a los grupos vecinos. Para su cálculo y visualización, se ha ejecutado el siguiente procedimiento:

```
#Create an array to store silhouette scores for different values of K
silhouette_scores = []
#Run K-means clustering for a range of K values and calculate the silhouette score
for each K:
for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=0)
    kmeans.fit(penguins_std)
    labels = kmeans.labels_
    silhouette_avg = silhouette_score(penguins_std, labels)
    silhouette_scores.append(silhouette_avg)
plt.figure(figsize=(8, 6))
plt.plot(range(2, 11), silhouette_scores, marker='o', linestyle='-', color='b')
plt.title('Silhouette Method')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Silhouette Score')
plt.grid(True)
plt.show()
```

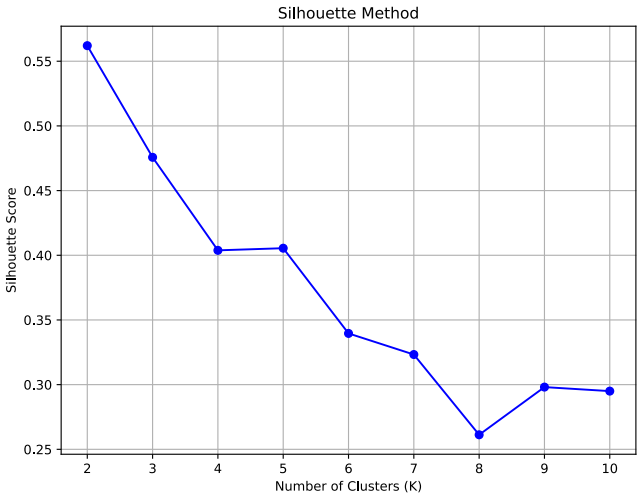


Fig.21 Puntuación de silueta según el número de grupos k

Al analizar el coeficiente de silueta, se observa que la puntuación máxima se alcanza con un valor de k=2. No obstante, con el fin de evitar una solución excesivamente simplista y capturar mejor la diversidad biológica del dataset, se ha optado por mantener una estructura de 3 grupos. Esta decisión permite una representación más fiel de la realidad de los datos, a pesar de la ligera disminución en la métrica de cohesión global.

```
plt.figure(figsize=(8, 6))
y_lower = 10
for i in range(4):
    ith_cluster_silhouette_values = silhouette_values[labels == i]
    ith_cluster_silhouette_values.sort()
    size_cluster_i = ith_cluster_silhouette_values.shape[0]
    y_upper = y_lower + size_cluster_i
    color = plt.cm.get_cmap("Spectral")(float(i) / 4)
    plt.fill_betweenx(np.arange(y_lower, y_upper),
                      0, ith_cluster_silhouette_values,
                      facecolor=color, edgecolor=color, alpha=0.7)
    plt.text(-0.05, y_lower + 0.5 * size_cluster_i, str(i))
    y_lower = y_upper + 10
plt.title("Silhouette Plot for Clusters")
plt.xlabel("Silhouette Coefficient Values")
plt.ylabel("Cluster Label")
plt.grid(True)
plt.savefig("sombras_silueta.svg",bbox_inches="tight")
plt.show()
```

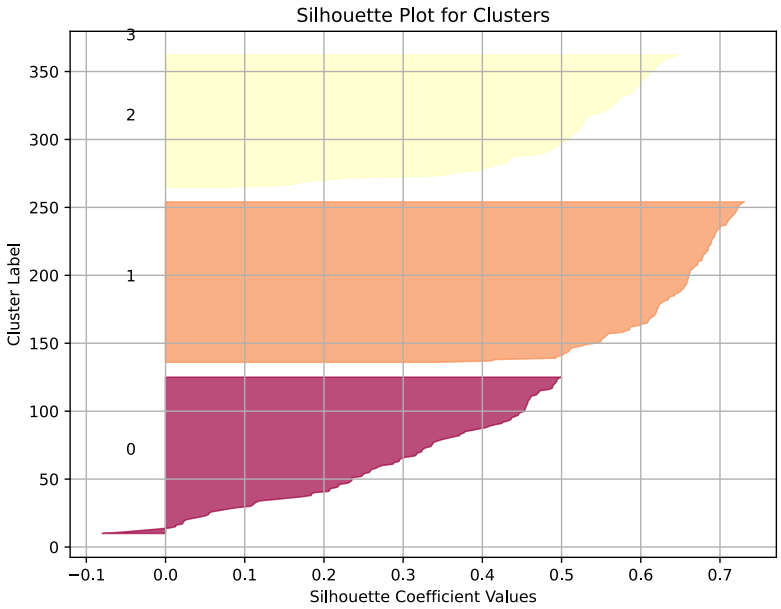


Fig.22 Puntuaciones de silueta para los 3 grupos

El análisis del gráfico de silueta de la figura 22 revela una calidad de agrupamiento heterogénea.

Los Clústeres 1 y 2 presentan los mejores indicadores de cohesión, con coeficientes de silueta consistentemente superiores a 0.5, lo que evidencia una estructura interna compacta y bien definida.

Por el contrario, el Clúster 0 se identifica como el grupo más problemático. Su perfil de silueta muestra una alta dispersión e incluye valores negativos, lo cual, en la teoría de este método, indica que ciertos individuos han sido asignados al clúster incorrecto. Esto corrobora matemáticamente la existencia de solapamiento en las fronteras de decisión, probablemente debido a la similitud fenotípica entre las especies Adelie y Chinstrap discutida anteriormente.

9. Jerárquico versus K-Means

Para cuantificar las diferencias entre ambos métodos, se ha generado una matriz de confusión cruzando las etiquetas asignadas por cada algoritmo (Figura 23). Se observa una coincidencia absoluta en la identificación del primer clúster (119 individuos), lo que confirma la robustez del modelo para aislar el grupo correspondiente a la especie Gentoo.

Las discrepancias se limitan exclusivamente a la frontera entre los otros dos grupos. Mientras que 116 y 90 individuos han sido clasificados de manera consistente por ambos modelos, existe un subconjunto de 8 observaciones donde los algoritmos difieren. Esto representa una muy elevada, lo que valida que la estructura detectada es estable y que las únicas dudas surgen en los casos límite de solapamiento entre Adelie y Chinstrap.

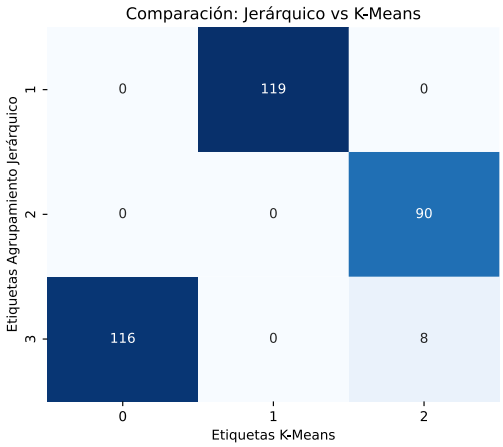


Fig.23 Matriz de confusión de los dos métodos de agrupamiento

10. Interpretación de los grupos

Tras analizar los resultados del agrupamiento y compararlos con las etiquetas reales de las especies, se observa que los tres grupos identificados presentan una correspondencia clara con diferencias morfológicas entre los pingüinos. Los resultados sugieren que el clustering no responde únicamente a una partición arbitraria del espacio de datos, sino que refleja estructuras coherentes con las características físicas descritas previamente mediante el análisis de

componentes principales.

El Grupo 1 corresponde al clúster más compacto y claramente diferenciado, coincidiendo con los individuos situados en valores positivos de la primera componente principal (PC1). Esta componente está fuertemente asociada al tamaño corporal, por lo que este grupo puede interpretarse como representativo de la especie Gentoo. Estos individuos presentan mayores valores de masa corporal y longitud de aleta, lo que facilita su separación respecto al resto del conjunto de datos.
Los Grupos 2 y 3 se localizan principalmente en la región negativa de la PC1, asociada a individuos de menor tamaño corporal, e incluyen a las especies Adelie y Chinstrap. La diferenciación entre ambos grupos se relaciona principalmente con la segunda componente principal (PC2), vinculada a variaciones en la morfología del pico. No obstante, la separación entre estas especies resulta menos nítida, observándose un solapamiento parcial entre individuos, lo que indica que sus características físicas presentan mayor similitud relativa.

En conjunto, la estructura de tres grupos seleccionada resulta coherente con la presencia de las tres especies analizadas. Sin embargo, el análisis pone de manifiesto una diferencia notable en la separabilidad de los grupos: mientras que la especie Gentoo muestra un perfil morfológico claramente diferenciado, las especies Adelie y Chinstrap presentan fronteras más difusas, lo que se refleja en valores de cohesión menores y en la dificultad de los métodos no supervisados para establecer una división completamente definida entre ambas.

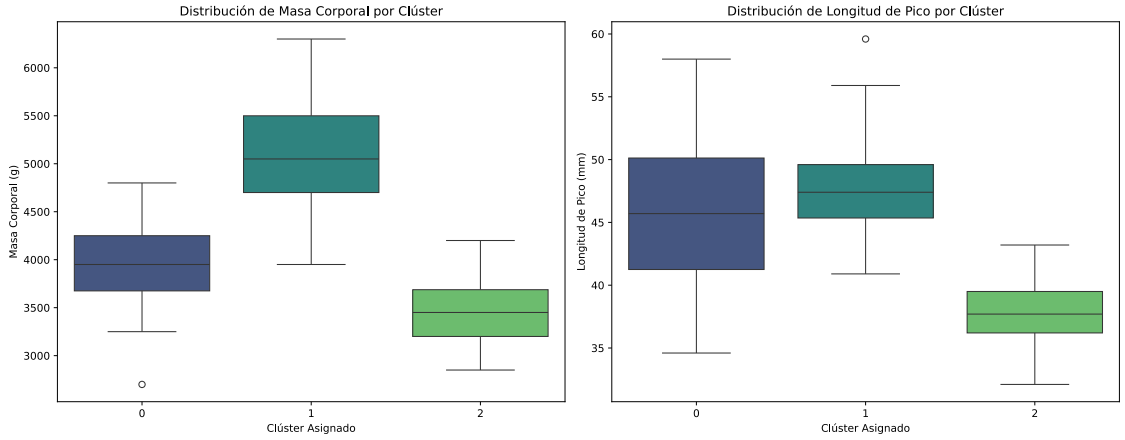


Fig.24 Boxplot de características originales por grupo

Para corroborar esta interpretación biológica, se presenta en la Figura 24 la distribución de las variables originales según el clúster asignado. Se aprecia claramente cómo el Clúster 1 (asociado a *Gentoo*) presenta una mediana de masa corporal significativamente superior (ver gráfico izquierdo). Por su parte, la distinción entre los Clústeres 0 y 2 (asociados a *Adelie* y *Chinstrap*) se hace evidente en la variable de longitud del pico (ver gráfico derecho), donde uno de los grupos muestra valores consistentemente más altos, validando que el algoritmo ha capturado las sutiles diferencias morfológicas del pico entre estas dos especies.

11. Resumen de hallazgos y análisis

El análisis realizado sobre el conjunto de datos Palmer Penguins ha permitido identificar patrones latentes en la morfología de las especies mediante técnicas de reducción de dimensionalidad y aprendizaje no supervisado. En conjunto, los resultados muestran que tanto el análisis de componentes principales (PCA) como los métodos de agrupamiento jerárquico y K-Means revelan una estructura de datos organizada en tres grupos principales, coherente con la

clasificación biológica de las especies Adelie, Chinstrap y Gentoo.

La primera componente principal, asociada principalmente al tamaño corporal global, especialmente masa y longitud de las aletas, emerge como la principal variabilidad dominante del conjunto de datos. Gracias a esta dimensión, la especie Gentoo se diferencia claramente del resto, formando un clúster compacto y bien definido. Por el contrario, las especies Adelie y Chinstrap presentan una mayor proximidad en el espacio de características, ya que comparten dimensiones corporales similares. Su separación depende principalmente de la segunda componente principal, relacionada con variaciones más sutiles en la morfología del pico, lo que genera una zona de solapamiento parcial y explica la mayor dificultad de los algoritmos no supervisados para establecer una frontera completamente nítida entre ambas.

A pesar de las diferencias conceptuales entre el agrupamiento jerárquico y K-Means, ambos enfoques producen resultados altamente consistentes, lo que refuerza la estabilidad de los patrones detectados. Asimismo, el análisis de validación pone de manifiesto la importancia de combinar criterios cuantitativos y contexto interpretativo: aunque la puntuación de silueta sugiere una solución más simple con $k = 2$, la elección de $k = 3$ ofrece una representación más informativa y coherente con la diversidad morfológica observada.

En conjunto, este trabajo evidencia que la integración de PCA y técnicas de clustering constituye una herramienta eficaz para explorar estructuras latentes en datos biológicos, permitiendo interpretar la variabilidad morfológica desde una perspectiva estadística rigurosa sin perder el contexto interpretativo del fenómeno estudiado.

